Week 5 Practice: Building a Single-Page App

Oct 13, 2026 · @thavorac chun

In 2 hours you turn Campus Events into a real single-page app: pages with Vue Router, shared state with Pinia, a validated booking form, and data from a mock API. Then you do the same for your own app, which is milestone M1, due at the end of this week. Starter code: `week05-starter.zip`.

Setup (10 min)

Unzip the starter into `week05/` of your practice repository. The app now needs two processes, so open two terminals.

```
cd week05/campus-events-week05
npm install
npm run api     # terminal 1: json-server on http://localhost:3000
npm run dev     # terminal 2: the app on http://localhost:5173
npm test        # 29 tests; they fake fetch, so the API need not run
```

1. Open http://localhost:3000/events in the browser. This is the JSON the app will load.
2. Open the app. The list says “Loading...” forever: the events store does not fetch yet.
3. Read what is already done for you: the API layer (`src/api/`), the pages (`src/pages/`), `EventForm.vue` (a finished form to copy from) and the session store (the role switch in the header).

In class, finish Parts A to C. Finish Parts D and E at home before you start your own M1 work.

Part A: Routes, params and a guard (20 min)

Open `src/router/index.ts`.

1. Add the four routes listed in the TODO. Lazy-load each page with `component: () => import("../pages/....vue")`.
2. The detail route is `/events/:id(\\d+)` with `props: true`, so `EventDetailPage` receives the id as a prop. Only digits match: `/events/abc` goes to the 404 page.
3. The catch-all `/:pathMatch(.*)*` goes last.
4. Write the `router.beforeEach` guard: if `to.meta.organiserOnly` is set and the session is not an organiser, redirect to `{ name: "events", query: { denied: "1" } }`.

Check in the browser: click an event title, use Back and Forward, refresh on `/events/2`, visit `/nope`. As a Student, type `/events/new` in the address bar; then switch the role to Organiser and try again.

`tests/router.test.ts` (3 tests) passes when you are done. Run `npm run build` and find one small `.js` file per lazy page in `dist/assets/`.

Part B: The events store (15 min)

Open `src/stores/events.ts`.

1. `byId`: a computed that returns a function `(id) => the event with that id, or undefined`.

2. `fetchAll`: set `status` to `"loading"` and `error` to `null`; call `eventsApi.list()`; on success store the events and set `"success"`; on failure store the error's message and set `"error"`.

Now the list loads. Test all four states that `EventsPage.vue` handles:

State	How to see it
Loading	DevTools, Network tab, throttle to Slow 3G, reload
Error	Stop `npm run api`, reload, then start it again and click Try again
Empty	Temporarily make `"events": []` in `db.json`
Success	Normal run

Open the Pinia tab in Vue DevTools and watch `status` change while the page loads. `tests/stores.test.ts` checks the events store (4 tests).

Part C: Validation and the booking form (25 min)

Read `EventForm.vue` and `validateEvent` first: they are the finished pattern. Then build the booking version.

1. In `src/validation.ts`, write `validateBooking(form, seatsLeft)`. The TODO lists each rule and its exact message. Run `npx vitest run tests/validation.test.ts` until it passes.
2. In `src/components/BookingForm.vue`:
    a. A `reactive` form object `{ name: "", email: "", seats: 1 }`, bound with `v-model` (seats with `v-model.number`).
    b. `submitted` (a ref) and `errors` (a computed that calls `validateBooking`).
    c. `onSubmit`: set `submitted`; stop if there are errors; otherwise emit `submit` with the trimmed name and email.
    d. Under each field, a `<p class="error">` that shows only after the first submit.
    e. While `pending`, disable the button and show "Booking...".

Try it: submit the empty form, then fix one field at a time and watch its message disappear. Book 4 seats on an event that has 3 left.

The server must check all of this again. In week 6 you will send a booking with `curl` that skips this form entirely.

Part D: The booking action (15 min, finish at home)

Open `src/stores/bookings.ts` and write `book(event, input)`:

1. If `input.seats` is more than the seats left, throw `new ApiError(409, "Not enough seats left.")` without calling the API.
2. POST the booking with `bookingsApi.create`, including `eventId` and `createdAt`.
3. PATCH the event's new seat count with `eventsApi.setBooked`.

4. Put the updated event in the events store with `useEventsStore().replace(...)`, add the booking at the start of `bookings`, and return it.

Book a seat in the browser. You land on My bookings, the header shows the seat count, and `db.json` now contains your booking. Cancel it and check that the seats come back. `git checkout db.json` resets the data.

Part E: A composable (10 min, finish at home)

Move the filtering logic from week 4's `EventList.vue` into `src/composables/useEventFilters.ts` and return `{ search, category, visible }`. `EventList.vue` already calls it.

You are done when `npm test` shows 29 of 29 passing, `npm run check` shows no errors, and you can book a seat and create an event (as Organiser) in the browser.

Part F: Your app, M1 frontend prototype (rest of the session, then at home)

Apply the same steps to your project, on a branch, merged by pull request.

1. Shape `db.json` like the API you will build in weeks 6 and 7: one array per entity, with ids that link them (for example `"eventId": 2`).
2. Add `json-server` and an `api` script; copy `src/api/http.ts` and write one file per resource in `src/api/`.
3. Routes for your main pages, a detail page with a param, and a 404 page. Add a guard if you have pages for one role.
4. At least one Pinia store whose pages handle loading, error, empty and success.
5. At least one form with validation and clear messages under each field.
6. A README section: how to run it (`npm run api`, `npm run dev`).

Submit by pushing the tag `m1`:

```
git tag m1
git push --follow-tags
```

M1 is checked in the week 6 practice session. Be ready to show the four states of one page, explain why a piece of state is in a store, and say where a field is validated.

Explain it

Answer in `week05/ANSWERS.md`, 2–4 sentences each.

1. You refresh the browser on `/events/2` and the page loads. Which program answered that request in development, and what must the production server do so it still works after deployment?
2. The guard stops students from opening `/events/new`. Can a student still create an event? How, and what must stop them?
3. `EventDetailPage` loads its event in a `watch` on the id with `immediate: true`, not in `onMounted`. What bug would `onMounted` alone cause?
4. Why does `EventsPage` use `storeToRefs(store)` instead of `const { events } = store`?

5. Booking makes two requests: POST the booking, then PATCH the seat count. What happens to the data if the second one fails? How could the server avoid this?

Checkpoint before you leave

- ☐ Parts A to C done; `tests/router.test.ts` and the events store tests pass
- ☐ You have seen all four states of the events page
- ☐ Your app has its routes and its `db.json`, running against json-server
- ☐ At home: Parts D and E (29 of 29 tests), then M1 tagged `m1`
- ☐ `week05/ANSWERS.md` answers the five questions

Next week: HTTP and REST in depth, then NestJS. You write your API contract and replace json-server with your own backend.